TrustFactor System Documentation

Two-Factor Authentication and Account Management Platform

1. Introduction

TrustFactor is a web-based authentication platform designed to improve account security through two-factor authentication, protected session handling, and administrator account management. The system allows users to register, sign in, configure a second verification method, and manage access securely. Administrators can monitor sessions, user activity, verification events, and account status from a dedicated dashboard.

The current implementation uses a Node.js and Express backend, an SQLite database, and a single-page frontend built with HTML, CSS, and JavaScript. The platform has been updated with TrustFactor branding, a redesigned interface, dark mode support, and improved usability for both regular users and administrators.

2. Project Overview

The purpose of TrustFactor is to provide a simple but secure authentication system for academic, prototype, or internal-use deployments. It combines login protection, two-factor verification, account lockout controls, rate limiting, and session management in one application.

The system supports two verification methods:

1. Authenticator App verification using time-based one-time passwords (TOTP)
2. Email Code Delivery for secondary verification

The system no longer uses SMS verification, and developer sandbox logs have been removed from the live product flow.

3. 4Objectives

The main objectives of TrustFactor are:

1. To secure user logins with optional two-factor authentication
2. To support modern verification methods such as authenticator apps and email-based codes
3. To prevent abuse using account lockouts and IP-based rate limiting
4. To provide administrators with user monitoring and account management tools
5. To offer a clean, modern, and usable interface in both light mode and dark mode
6. To maintain session security through token hashing and controlled expiration

4. Scope and Limitations

Scope:

1. User registration and sign-in
2. Session creation, validation, and sign-out
3. 2FA setup, verification, and disable flow
4. Backup recovery code generation
5. Admin dashboard for metrics and account management
6. Username and password editing for managed accounts

Limitations:

1. Email delivery depends on valid Gmail SMTP credentials in environment settings
2. SQLite is suitable for small to medium deployments but may not be ideal for high-scale

production systems
3. The current test script is only a smoke placeholder and not a full automated test suite
4. Theme preference is stored in the browser, not in the database

5. System Architecture

TrustFactor follows a client-server architecture:

1. Frontend Layer
   - Built with HTML, CSS, and vanilla JavaScript
   - Handles view switching, forms, 2FA setup wizard, dark mode, and admin UI rendering
2. Backend Layer
   - Built with Node.js and Express
   - Handles API requests, session validation, login flow, verification logic, and admin operations
3. Database Layer
   - Uses SQLite for persistent storage
   - Stores users, sessions, verification codes, secrets, and backup code hashes
4. Security Layer
   - Uses bcrypt for password hashing
   - Uses SHA-256 for token and backup code hashing
   - Uses AES-256-GCM encryption for stored TOTP secrets

6. Core Features and Functional Requirements

6.1 User Registration

Users can create an account using a username, email address, and password. The system validates input and prevents duplicate usernames or emails.

6.2 Login Flow

Users log in using either username or email plus password. If 2FA is disabled, login completes immediately. If 2FA is enabled, the user is redirected to the verification step.

6.3 Two-Factor Verification

When 2FA is enabled, TrustFactor supports:

1. Authenticator App verification
2. Email code verification
3. Backup recovery code login

6.4 2FA Setup Wizard

Users can enable 2FA from the dashboard through a guided setup wizard. For authenticator apps, the system generates a QR code and secret key. For email verification, the system sends a confirmation code to the registered email.

6.5 Account Security Controls

The system applies lockouts after too many failed login or 2FA attempts. IP-based rate limiting is also enabled, with successful authentication requests excluded from the auth counter.

6.6 Admin Dashboard

Administrators can view:

1. User metrics
2. 2FA-enabled accounts

3. Active sessions
4. Locked accounts
5. Verification activity
6. User management table

6.7 Manage Accounts

Admins can create users, delete users, and edit user credentials. Current edit support is limited to username and password, based on the latest system update.

7. Non-Functional Requirements

1. Security: Password hashing, encrypted TOTP secrets, secure cookies, and session expiration
2. Performance: Lightweight SQLite-backed system appropriate for local or small deployments
3. Usability: Responsive modern UI with light and dark themes
4. Maintainability: Clear backend separation between database, security, and services modules
5. Reliability: Input validation and controlled error handling for authentication workflows

8. Database Design

TrustFactor uses the following main tables:

users

Stores account identity, password hash, role, 2FA status, TOTP secret, backup codes, lockout data, and timestamps.

sessions

Stores active session records, token hashes, IP addresses, device information, expiration times, and creation times.

two_fa_codes

Stores generated verification codes for email delivery, including code hash, expiry, attempt count, and usage state.

Important stored fields include:

```
users.user_id
users.username
users.email
users.password_hash
users.role
users.two_fa_enabled
users.two_fa_method
users.totp_secret
users.backup_codes
users.lockout_until
sessions.token_hash
two_fa_codes.code_hash
```

9. User Interface Description

The TrustFactor interface consists of the following views:

1. Registration View
2. Login View

3. 2FA Verification View
4. User Dashboard
5. Admin Dashboard

The user dashboard provides profile details, 2FA status, enable/disable actions, and the setup wizard. The admin dashboard provides summary cards, security activity tables, session monitoring, and account management.

The system also includes a dark mode toggle for all users. The chosen theme is stored in browser local storage to preserve appearance between visits on the same device.

10. Security Features

TrustFactor includes several security features:

1. bcrypt password hashing
2. AES-256-GCM encryption for TOTP secrets
3. SHA-256 hashing for session tokens and backup codes
4. Secure cookie-based session handling
5. Account lockout after repeated failures
6. IP-based request throttling
7. Backup recovery codes for account access recovery
8. Role-based admin access control

11. Setup and Deployment Procedure

To run the system locally:

1. Install Node.js
2. Open the project folder
3. Install dependencies using: `npm install`
4. Start the application using: `node server.js` or `npm start`
5. Open the system in a browser at: `http://localhost:3000`

Environment variables may be configured through the `.env` file, including:

```
PORT
SESSION_TIMEOUT_MINUTES
MAX_LOGIN_ATTEMPTS
LOCKOUT_DURATION_MINUTES
CODE_EXPIRATION_MINUTES
GMAIL_USER
GMAIL_APP_PASSWORD
ENCRYPTION_KEY
```

12. Testing and Validation

The project currently includes a smoke placeholder test file rather than a complete automated test suite. Manual verification should cover:

1. Registration success and duplicate prevention
2. Login with correct and incorrect credentials
3. Account lockout behavior
4. Authenticator setup and verification
5. Email verification flow
6. Backup code login

7. Session validation and sign-out
8. Admin account editing and deletion rules
9. Light mode and dark mode UI readability

13. Current Enhancements Applied to This Version

This TrustFactor version includes the following completed improvements:

1. Removed SMS verification from the live system
2. Removed sandbox log features from the live system
3. Renamed the platform to TrustFactor
4. Redesigned the interface with a clean modern look
5. Added dark mode toggle support for all users
6. Improved dark mode visibility in the dashboard and admin sections
7. Fixed sign-out behavior
8. Fixed admin manage-account editing for username and password
9. Increased IP request limits and excluded successful logins from auth rate-limit counting

14. Recommendations for Future Development

1. Add a full automated test suite for backend and frontend behavior
2. Add audit logs for administrator actions
3. Support password reset through secure email links
4. Move from SQLite to PostgreSQL or MySQL for larger deployments
5. Store theme and profile preferences at the account level
6. Add exportable reports for admin activity monitoring

15. Conclusion

TrustFactor is a secure and user-friendly authentication system designed to strengthen account protection using two-factor verification, secure sessions, and centralized administration tools. Its current architecture is suitable for demonstration, academic submission, or small-scale deployment, while also providing a strong foundation for future improvements and production-oriented expansion.

16. Appendix: Main Project Files

```
server.js - Main backend server and API routes
db.js - Database initialization and query helpers
security.js - Cryptographic and authentication helpers
services.js - Email delivery service
public/index.html - Frontend structure
public/css/style.css - User interface styling and theme support
public/js/app.js - Frontend logic and dashboard behavior
```